LAB-2 -ACTIVITY

Name:	
CMS:	
Date	

PROGRAM STRUCTURE

Program:

It is a set of instructions in computer language that perform a specific task.

Structure:

A structure is an arrangement and organization of interrelated elements in a material object or system, or the object or system so organized.

Assembly Language Program Structure:

A processor can directly execute a machine language program. Though it is possible to program directly in machine language, assembly language uses mnemonics to make programming easier. Every program has 2 attributes. An assembly language program also uses mnemonics to represent:

- Symbolic instructions.
- The raw data that represents variables and constant.

A machine language program consist of:

- A list of numbers representing the bytes of machine instruction to be executed.
- Data constants to be used by the program.

A program has always the following general structure:

```
.model      small          ;Select a memory model
.stack      100h           ;Define the stack size
.data
            ; Variable and array declarations
            ; Declare variables at this level
.code
main        proc
      ; Write the program main code at this level
main        endp
      ;Other Procedures
      ; Always organize your program into procedures
end         main           ; To mark the end of the source file
```

What (.) shows here?

Dot (.) is used to access predefined directives.

Directives of Assembly Language:

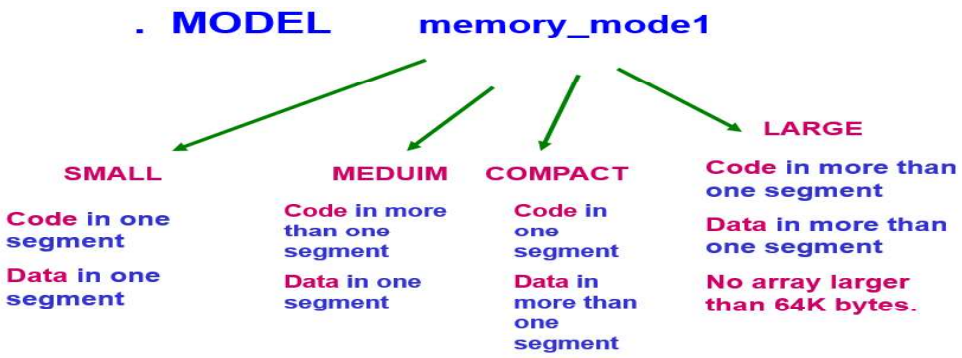
1. **.MODEL**
2. **.STACK**
3. **.DATA**
4. **.CODE**

Use of Directives of Assembly Language:

Program Model:

The size of the **code & data** a program can have is determined by specifying a memory model using the **.MODEL** directive.

Syntax



Memory Models

Model	Description
tiny	A single segment, containing both code and data. This model is used by .com programs.
small	One code segment and one data segment. All code and data are near, by default.
medium	Multiple code segments and a single data segment.
compact	One code segment and multiple data segments.
large	Multiple code and data segments.
huge	Same as the large model, except that individual data items may be larger than a single segment.
flat	Protected mode. Uses 32-bit offsets for code and data. All data and code (including system resources) are in a single 32-bit segment.

- **Unless there is a lot of code or data, the appropriate model is SMALL.**
- . MODEL **directive should come before any segment definition.**

Stack Segment:

- Used to set aside storage for the stack.
- Stack addresses are computed as offsets into this segment.
- Use: **.stack** followed by a value that indicates the size of the stack.

Declaration Syntax

.STACK size

Size is an optional number that specifies the stack area size in bytes.

.STACK 100H

- Sets aside 100h bytes for the stack area (a reasonable size for most applications).
- If size is omitted, 1 KB is set aside for the stack area.

Data Segment:

- A program's **data segment** contains all the variable definitions. Constant definitions are made here as well, but they may be placed elsewhere in the program since no memory allocation is involved.
- We use the **. DATA directive** followed by variable & constant declarations.
- Variable addresses are computed as offsets from the start of this segment
- It is optional, if we want to use external memory then we can use it.

Example

```
.DATA
WORD1    DW  2
WORD2    DW  5
MSG      DB  ' This is a message '
MASK     EQU 10010010B
```

Code Segment:

- It contains a program's instructions.

Syntax

```
.CODE    name
```

- Name is optional for the segment, there is no need to name the small program.
- Inside the code segment instructions are organized as procedures

Simplest procedures Definition:

```
name     PROC
; body of the procedure
name     ENDP
```

- Here name is the name of the procedure.
- **PROC** and **ENDP** are pseudo-op that delineate the procedure

EXAMPLE

```
.CODE
    MAIN PROC
    ; main procedure body
    MAIN ENDP
    ; other procedures go here
```

Assembly Language DATA Types:

- byte – 8 bits.
- word – 16 bits.
- qword – 32 bits.
- dword – 64 bits.
- Real4 – 32 bit float.
- Real8 – 64 bit float.
- Real10 – 80 bit float.

Simple program in Assembly Language:

1. **Printing a single character on to screen:**

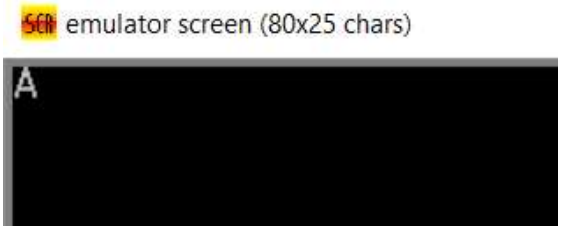
CODE:

```
;first proram to display a character on screen
.model small
.stack 100h
.code
main proc
        mov ah,02h
        mov dl,'A'
        int 21h

        mov ah,4ch
        int 21h

main endp
end main
```

OUTPUT:



2. **Program to display 'IBA' on the Screen.**

Code:

```
;first proram to display 'IBA'on screen
.model small
.stack 100h
.code
main proc
        mov ah,02h
        mov dl,'I'
        int 21h

        mov dl,'B'
        int 21h

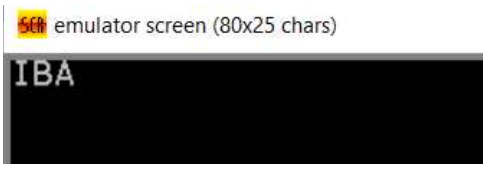
        mov dl,'A'
        int 21h

        mov ah,4ch
        int 21h

main endp

end main
```

OUTPUT:



Name:	
CMS:	
Date	

SERVICE NUMBERS

02h:
it is used to display single character on screen.
It is stored in 'ah' register.

01h:

Used to take single character as input from keyboard.
It is stored in 'al' register.

09h:

Used to display string on screen.
It is stored in 'ah' register.

RULES OF MICROPROCESSOR:

RULE:1 **Display single character on screen**

- **service number 02h must be stored in 'ah'.**
- **Invoke interrupt 21h(for input/output operations).**
- **Character must be stored in 'dl' register.**

RULE:2 **Get input as single character from keyboard.**

- **Service number 01h.**
  Service number will be stored in 'ah'.
- **Character will be stored in 'al' register by processor.**
- **Invoke interrupt 21h for operation.**

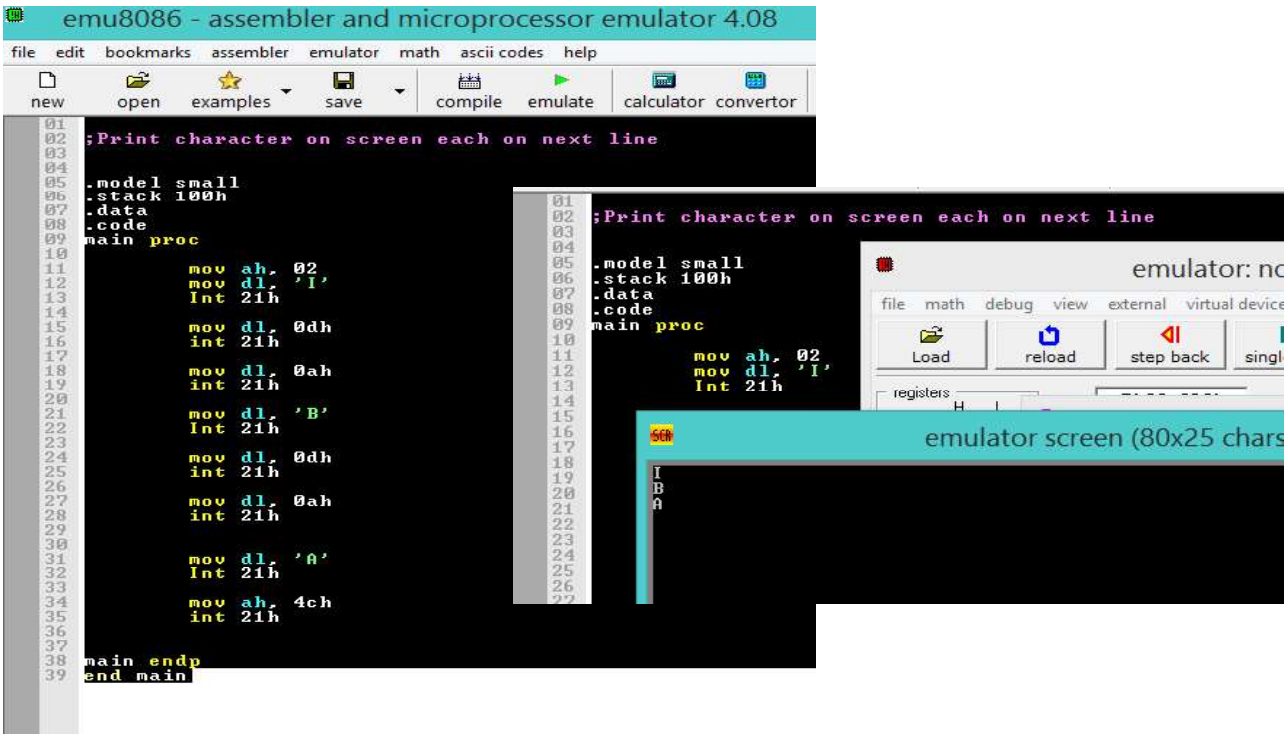
RULE:3 **Display string value on the screen.**

- **Service number 09h for storing input in register.**
- **It must be stored in ah register.**
- **Invoke interrupt 21h.**
- **Place base address in 'dx' register using 'lea' keyword.**
- **.data will be used for this operation.**

LAB TASKS

Task 01:

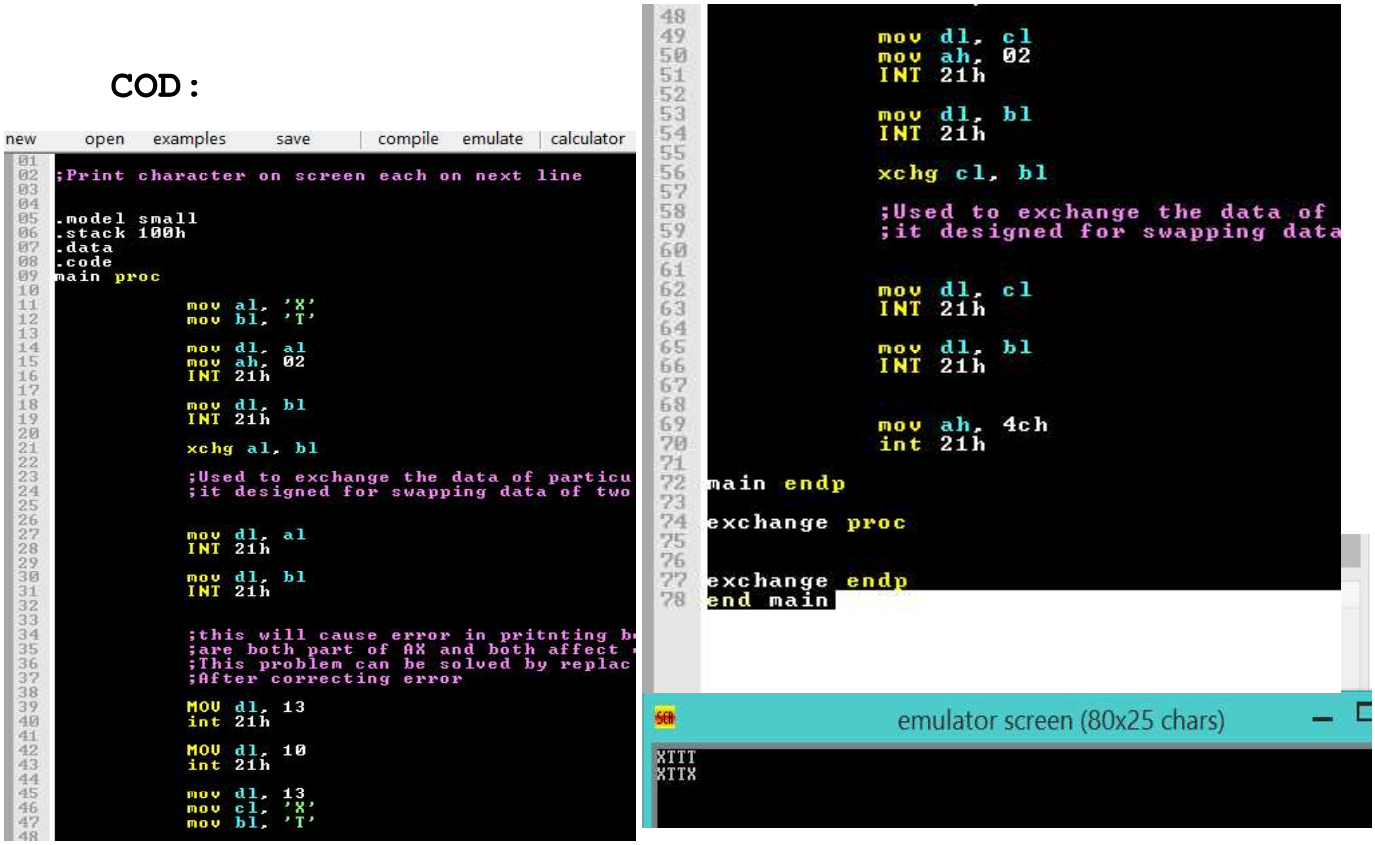
Printing IBA (each character on next line):



Task02

Write a program to swap two numbers or characters

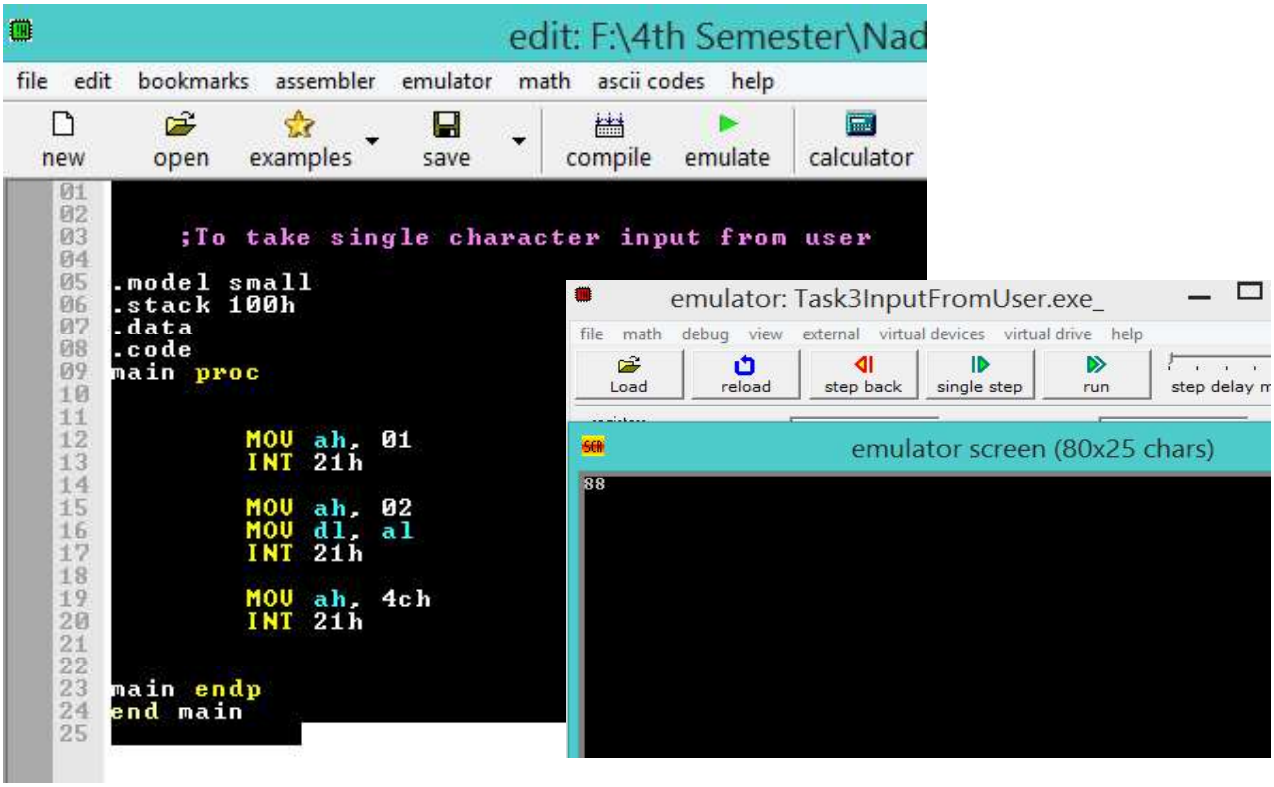
COD :



Task03:

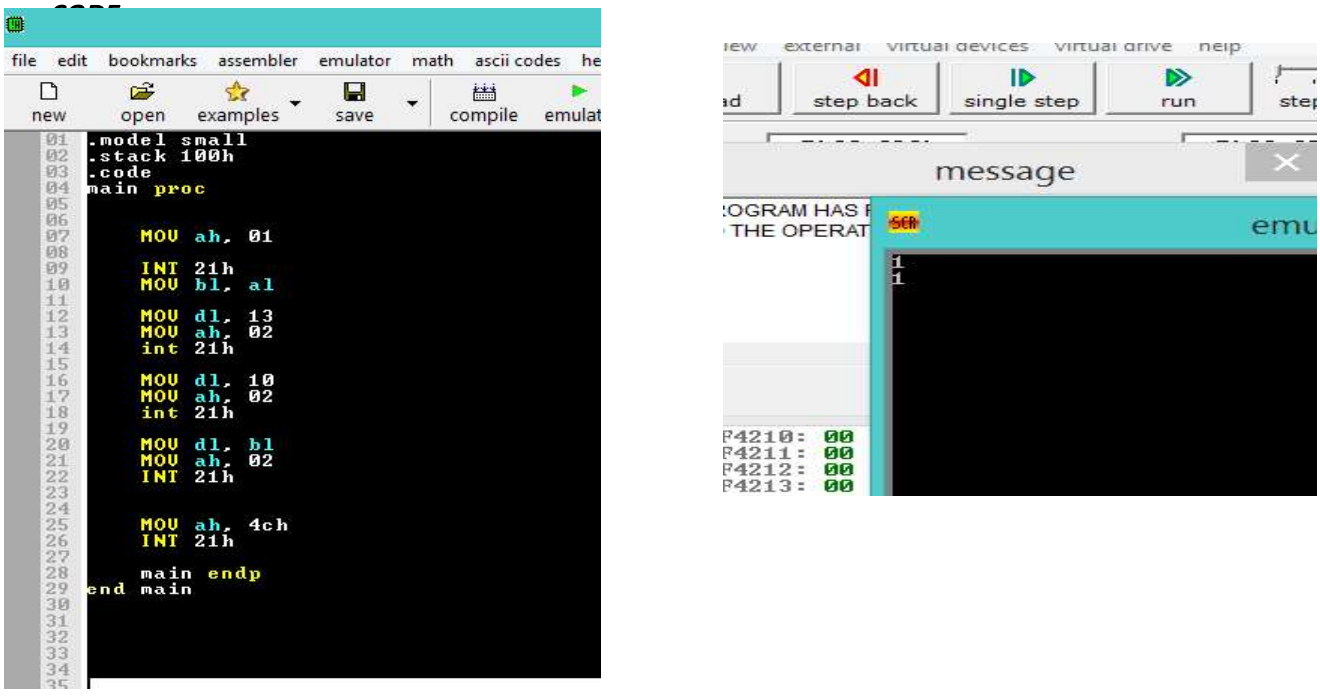
Take a single character input from user and display on screen.

CODE:



Task 04:

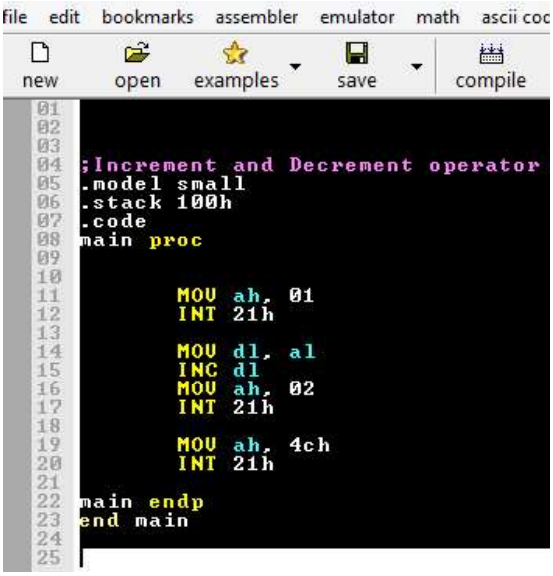
Take a single character input from user and display on next line.



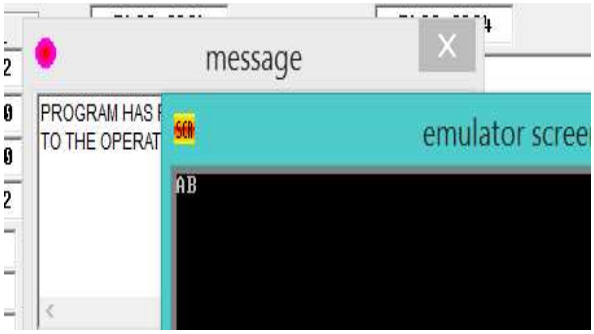
Task 05:

Input character from User and print its next.

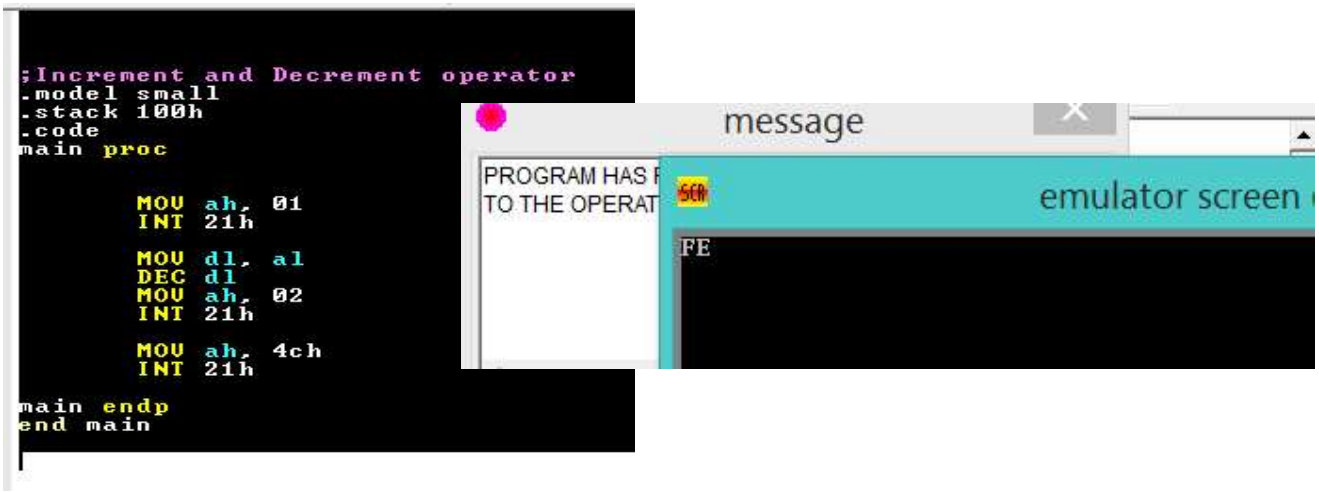
CODE:



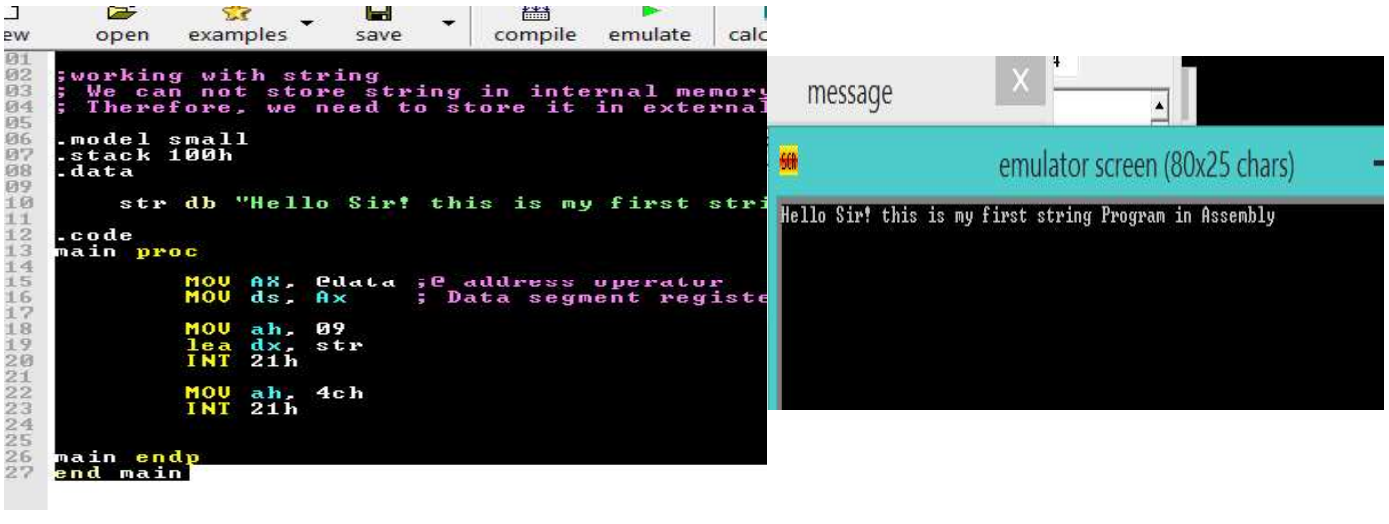
Output:



Task 06: ***Take input from user and print its previous characters.***



Task 07: ***Store and Display String on screen using AL program***



Name:	
CMS:	
Date	

PRORAM STRUCTURE IN AL

VARIABLE

signed : positive and negative numbers

unsigned: only positive numbers

DATA TYPES

DB(Define byte)

Unsigned

- 1 byte of size
- range (0-255)

Signed

- range(-128 to 127)

DW(Define word)

Unsigned

- 2 bytes of size
- range (0-65535)

EQU datatype

it is used to creating constants in **.data** model

Ex: Mask EQU 01000001b

here Mask is variable name, EQU is data type, and 01000001b

is value of 65(A).